

4. ระบบจองโรงแรม/ที่พัก (Hotel Booking System)

Allows users to search for available rooms, make reservations, and complete payments.

Main scenario:

The same room is requested simultaneously from multiple booking channels.

System scope:

- Covers room availability, reservation, locking, and booking confirmation.
- Does not include internal hotel management systems.

สมาชิก

1.นายธิปไตย ศิริรักษ์ 6609650434	GitHub: 6609650434
2.นายพุฒิพงศ์ กำเนิดพงศ์ปรีดา 6609650541	GitHub: puttipong-6609650541
3.นางสาวธนัชพร วงศ์ตลาด 6609650012	GitHub: tanutchaphon_6609650012
4.นางสาวมนัสนันท์ อุดมรัตน์ 6609650608	GitHub: manatsanorth
5.อานุภาพ อนุรักษ์สยาม 6609650756	GitHub: Arnuparp_6609650756

1.System Requirements:

1.1 Functional Requirements

- 1.1 Search & Filter: ค้นหาห้องพักว่างตามวันที่ระบุ (Check-in/Out) และจำนวนผู้เข้าพัก
- 1.2 Real-time Availability: แสดงสถานะห้องว่างที่อัปเดตล่าสุด (ดึงจาก RoomInventory)
- 1.3 Room Reservation: สามารถจองห้องพักเพื่อเข้าสู่สถานะ Pending Lock (กักห้อง 15 นาที)
- 1.4 Payment: ชำระเงินผ่านช่องทางต่างๆ (QR, Credit Card) และรับสถานะการชำระเงิน
- 1.5 Booking History: ดูประวัติการจองและสถานะปัจจุบัน (Confirmed, Cancelled, Expired)
- 1.6 Notification: รับการแจ้งเตือนยืนยันการจองผ่าน Email/SMS/Push Notification
- 1.7 Inventory Management: ปรับปรุงจำนวนห้องพักในระบบ
- 1.8 Room Status Update: เปลี่ยนสถานะห้อง (Maintenance, Cleaning) เพื่อส่งผลต่อ availableCount

1.2 Non-functional Requirements

1.2.1 Performance & Scalability

- Response Time: หน้าที่ค้นหาห้องพักควรตอบสนองภายใน < 2 วินาที แม้ในช่วงที่มี Traffic สูง (ใช้ Redis Cache ช่วย)
- High Concurrency: ระบบต้องรองรับการจองพร้อมกัน (Race Condition) ได้อย่างถูกต้อง 100% โดยใช้ Distributed Locking ใน Redis
- Horizontal Scalability: ทุก Microservice ต้องเป็น Stateless เพื่อให้สามารถขยายจำนวน Instance ผ่าน Load Balancer ได้ทันทีเมื่อมีโหลดเพิ่มขึ้น

1.2.2 Security

- Authentication & Authorization: ใช้ OAuth 2.0 / JWT ในการระบุตัวตน และกำหนดสิทธิ์การเข้าถึง API ผ่าน API Gateway
- Data Encryption: ข้อมูลสำคัญ (เช่น ข้อมูลลูกค้า, ยอดเงิน) ต้องเข้ารหัสขณะส่ง (TLS 1.3) และขณะจัดเก็บ (Encryption at rest)
- Network Protection: ใช้ WAF ป้องกันการโจมตีประเภท SQL Injection, Cross-Site Scripting (XSS) และ DDoS

1.2.3 Availability & Resiliency

High Availability : ระบบควรมี Uptime ไม่ต่ำกว่า 99% (ใช้ Database Replication และ Multi-instance services)

Fault Tolerance: หาก Notification Service ล่ม กระบวนการจองหลักต้องทำงานต่อได้ (ใช้ Message Queue เป็น Buffer)

Data Integrity: ข้อมูล Inventory และ Booking ต้องสอดคล้องกันเสมอ (Eventual Consistency ผ่าน Message Queue หรือ Distributed Transaction)

1.2.4 Observability

Centralized Logging: ระบบต้องส่ง Log ไปที่ ELK Stack เพื่อให้สามารถสืบค้นข้อผิดพลาดย้อนหลังได้

Real-time Monitoring: มี Dashboard (Grafana) เพื่อดูอัตราการจองสำเร็จ, อัตรา Error ของ Payment Gateway และ Latency ของแต่ละ Service

2. Use Case Diagram:

Use Case Diagram ของระบบจองห้องพักโรงแรม (Hotel Booking System) ถูกออกแบบขึ้นเพื่อแสดงขอบเขตการทำงานของระบบ รวมถึงบทบาทของผู้ใช้งานและระบบภายนอกที่มีส่วนเกี่ยวข้องกับกระบวนการจองห้องพัก โดยระบบนี้มี Actor หลักทั้งหมด 3 บทบาท ได้แก่ **User**, **Payment Gateway** และ **Email Service**

User เป็นผู้ใช้งานหลักของระบบ สามารถค้นหาห้องพักที่ว่าง ดูรายละเอียดของห้องพัก เลือกห้องพักที่ต้องการ สร้างรายการจอง ชำระเงิน และยกเลิกการจองได้ กระบวนการทำงานเริ่มจากผู้ใช้งานค้นหาห้องพักตามวันเข้าพัก วันออก และประเภทห้องพัก จากนั้นผู้ใช้งานสามารถดูรายละเอียดของห้องพัก เช่น ราคา จำนวนผู้เข้าพัก สิ่งอำนวยความสะดวก และเงื่อนไขการจอง ก่อนเลือกห้องพักเพื่อเข้าสู่กระบวนการสร้างรายการจอง

เมื่อผู้ใช้งานสร้างรายการจอง ระบบจะดำเนินการ Lock Room Hold เพื่อกันห้องพักไว้ชั่วคราว และป้องกันไม่ให้ผู้อื่นจองห้องพักเดียวกันในช่วงเวลาเดียวกัน การทำงานนี้มีความสำคัญต่อระบบจองโรงแรม เนื่องจากอาจมีผู้ใช้งานหลายคนค้นหาและกดจองห้องพักเดียวกันพร้อมกัน ดังนั้นระบบจึงต้องมี Use Case Handle Concurrent Requests เพื่อจัดการคำขอที่เกิดขึ้นพร้อมกัน และทำให้การจองเกิดขึ้นอย่างถูกต้อง ไม่เกิดปัญหาการจองซ้ำหรือจำนวนห้องพักติดลบ

หลังจากสร้างรายการจองแล้ว ผู้ใช้งานสามารถดำเนินการชำระเงินผ่าน Use Case Make Payment โดยระบบจะเชื่อมต่อกับ **Payment Gateway** ซึ่งเป็นระบบภายนอกที่ทำหน้าที่ประมวลผลการชำระเงิน เมื่อการชำระเงินสำเร็จ ระบบจะดำเนินการ Confirm Booking เพื่อเปลี่ยนสถานะรายการจองเป็นยืนยันแล้ว จากนั้นระบบจะเรียกใช้ **Email Service** เพื่อดำเนินการ Send Confirmation Email ส่งอีเมลยืนยันการจองให้ผู้ใช้เป็นหลักฐาน

นอกจากนี้ ระบบยังรองรับกรณียกเลิกหรือหมดเวลาการจอง โดย Use Case Cancel Reservation จะเกิดขึ้นเมื่อผู้ใช้งานต้องการยกเลิกการจอง ส่วน Use Case Release Room Lock จะเกิดขึ้นในกรณีที่ต้องคืนห้องพักเข้าสู่ระบบ เช่น ผู้ใช้ยกเลิกการจอง การชำระเงินไม่สำเร็จ หรือรายการจองหมดเวลาที่กำหนด ทำให้ห้องพักที่ถูกล็อกไว้สามารถกลับมาให้ผู้ใช้งานจองได้อีกครั้ง

ความสัมพันธ์ระหว่าง Use Case ถูกออกแบบให้สะท้อนลำดับการทำงานของระบบอย่างชัดเจน โดย Create Reservation มีความสัมพันธ์แบบ <<include>> กับ Lock Room Hold และ Handle Concurrent Requests เนื่องจากทุกครั้งที่มีการสร้างรายการจอง ระบบจำเป็นต้องล็อกห้องและตรวจสอบค่าขอที่เกิดขึ้นพร้อมกันเสมอ ส่วน Make Payment มีความสัมพันธ์แบบ <<include>> กับ Confirm Booking เพราะเมื่อชำระเงินสำเร็จ ระบบต้องยืนยันการจองต่อทันที และ Confirm Booking มีความสัมพันธ์แบบ <<include>> กับ Send Confirmation Email เนื่องจากระบบต้องส่งอีเมลยืนยันหลังจากการจองเสร็จสมบูรณ์ สำหรับ Cancel Reservation และ Release Room Lock เป็นความสัมพันธ์แบบ <<extend>> เนื่องจากเป็นกระบวนการที่เกิดขึ้นเฉพาะบางเงื่อนไขเท่านั้น

Actors และบทบาทที่เกี่ยวข้อง

Actor	บทบาทในระบบ
User	ผู้ใช้งานหลักของระบบ ทำหน้าที่ค้นหาห้องพัก ดูรายละเอียด เลือกห้อง สร้างการจอง ชำระเงิน และยกเลิกการจอง
Payment Gateway	ระบบภายนอกที่ใช้สำหรับตรวจสอบและประมวลผลการชำระเงิน
Email Service	ระบบภายนอกที่ใช้สำหรับส่งอีเมลยืนยันการจองหรือแจ้งสถานะให้ผู้ใช้

ตารางอธิบาย Use Case

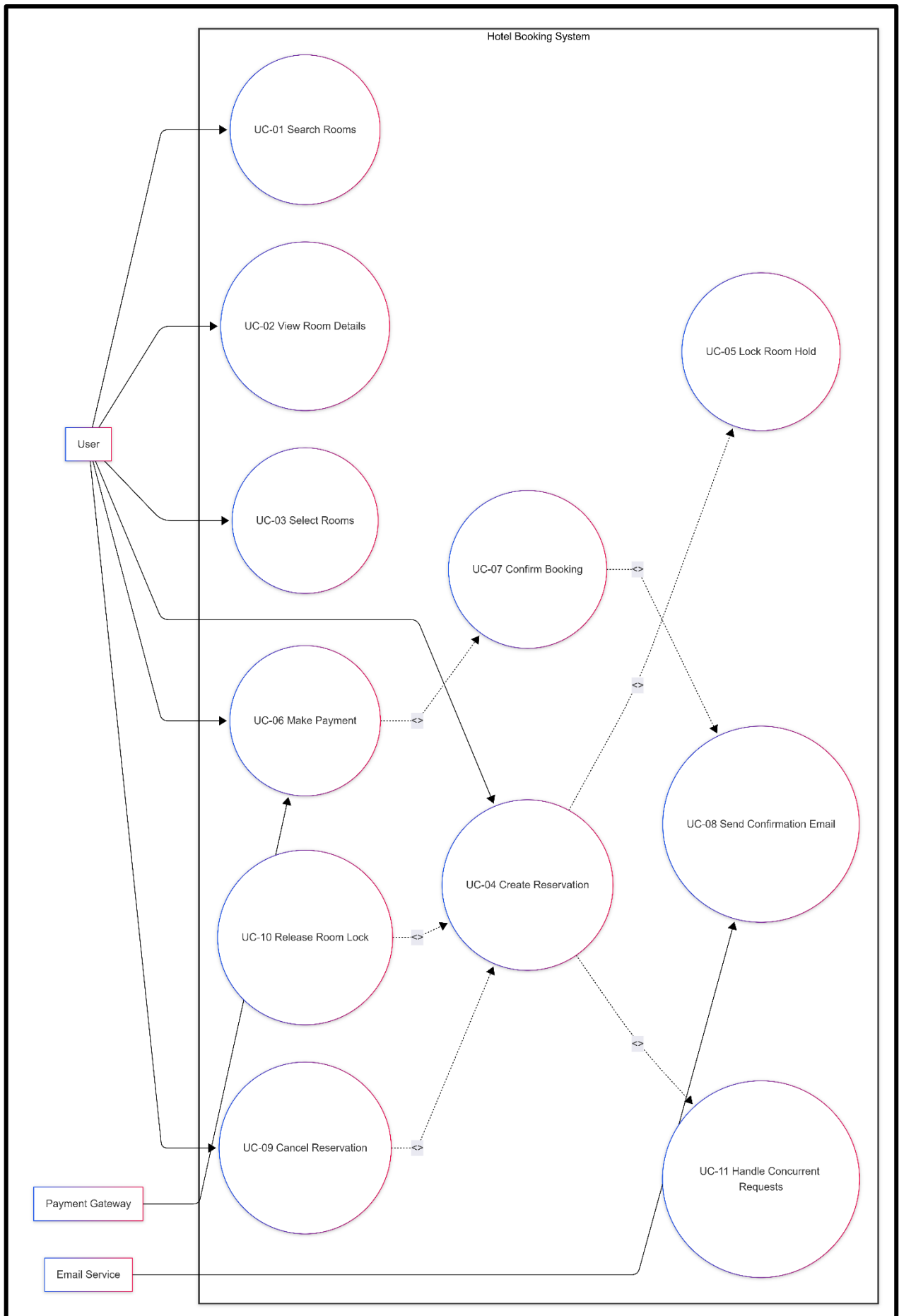
Use Case	รายละเอียด
UC-01 Search Rooms	ผู้ใช้งานหาห้องพักที่ว่างตามวันที่เข้าพัก วันที่ออก และประเภทห้อง
UC-02 View Room Details	ผู้ใช้งานดูรายละเอียดห้องพัก เช่น ราคา จำนวนผู้เข้าพัก สิ่งอำนวยความสะดวก และเงื่อนไขการจอง
UC-03 Select Rooms	ผู้ใช้งานเลือกห้องพักที่ต้องการจากรายการห้องที่ค้นหาได้
UC-04 Create Reservation	ผู้ใช้งานสร้างรายการจองห้องพัก โดยระบบจะบันทึกข้อมูลการจองเบื้องต้น
UC-05 Lock Room Hold	ระบบล็อกห้องพักชั่วคราว เพื่อป้องกันการจองซ้ำจากผู้อื่น
UC-06 Make Payment	ผู้ใช้งานชำระเงินค่าห้องพักผ่านระบบ โดยเชื่อมต่อกับ Payment Gateway
UC-07 Confirm Booking	ระบบยืนยันการจองเมื่อการชำระเงินสำเร็จ และเปลี่ยนสถานะการจองเป็นยืนยันแล้ว
UC-08 Send Confirmation Email	ระบบส่งอีเมลยืนยันการจองให้ผู้ใช้ เพื่อใช้เป็นหลักฐาน
UC-09 Cancel Reservation	ผู้ใช้งานยกเลิกการจองห้องพักในกรณีที่ไม่ต้องการดำเนินการต่อ
UC-10 Release Room Lock	ระบบปลดล็อกห้องพักเมื่อมีการยกเลิก ชำระเงินไม่สำเร็จ หรือหมดเวลาการจอง
UC-11 Handle Concurrent Requests	ระบบจัดการกรณีมีผู้ใช้หลายคนจองห้องเดียวกันพร้อมกัน เพื่อป้องกันการจองซ้ำ

ตารางความสัมพันธ์ระหว่าง Use Case

ความสัมพันธ์	คำอธิบาย
UC-04 Create Reservation <<include>> UC-05 Lock Room Hold	ทุกครั้งที่สร้างรายการจอง ระบบต้องล็อกห้องพักชั่วคราว
UC-04 Create Reservation <<include>> UC-11 Handle Concurrent Requests	ระบบต้องตรวจสอบคำขอที่เกิดขึ้นพร้อมกัน เพื่อป้องกันการจองซ้ำ
UC-06 Make Payment <<include>> UC-07 Confirm Booking	เมื่อชำระเงินสำเร็จ ระบบต้องยืนยันการจอง
UC-07 Confirm Booking <<include>> UC-08 Send Confirmation Email	หลังยืนยันการจอง ระบบต้องส่งอีเมลแจ้งผลให้ผู้ใช้
UC-09 Cancel Reservation <<extend>> UC-04 Create Reservation	การยกเลิกเกิดขึ้นเฉพาะกรณีที่ผู้ใช้เลือกยกเลิก
UC-10 Release Room Lock <<extend>> UC-04 Create Reservation	การปลดล็อกเกิดขึ้นเฉพาะกรณียกเลิก ชำระเงินไม่สำเร็จ หรือหมดเวลา

จาก Use Case Diagram ระบบจองห้องพักโรงแรมมีขอบเขตครอบคลุมกระบวนการตั้งแต่การค้นหาห้องพัก การดูรายละเอียด การเลือกห้อง การสร้างรายการจอง การล็อกห้องพักเพื่อป้องกันการจองซ้ำ การชำระเงิน การยืนยันการจอง การส่งอีเมลแจ้งผล และการจัดการกรณียกเลิกหรือหมดเวลาการจอง โดยระบบมีการเชื่อมต่อกับระบบภายนอก ได้แก่ Payment Gateway สำหรับประมวลผลการชำระเงิน และ Email Service สำหรับส่งอีเมลยืนยันผลการจอง ทำให้เห็นขอบเขตของระบบ บทบาทของผู้ใช้งาน และความสัมพันธ์ของแต่ละกระบวนการได้อย่างครบถ้วน

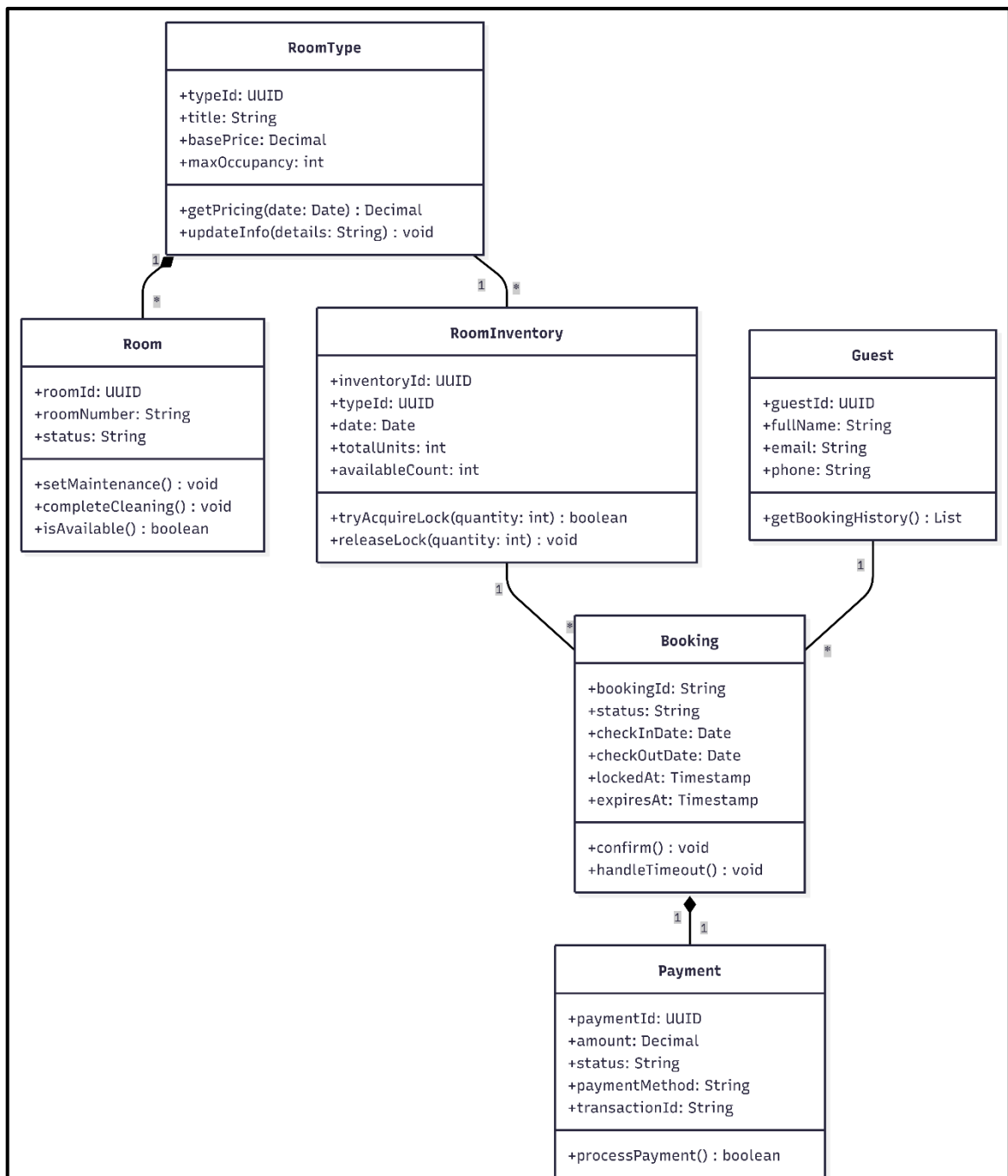
USECASE DIAGRAM



3.Data Model

- 1.RoomType (ประเภทห้อง): เพื่อระบุว่าลูกค้ากำลังจะจองห้องแบบไหนและราคาเท่าไร
- 2.RoomInventory (ตัวควบคุมจำนวนห้องพัก): ใช้ตรวจสอบและกัก (Lock) จำนวนห้องว่างในวันที่ระบุ
- 3.Booking (รายการจอง): เก็บสถานะการจอง เวลาที่เริ่มกักห้อง และระยะเวลาหมดอายุ
- 4.Guest (ผู้จอง): ข้อมูลลูกค้าที่ถือครองสิทธิ์การกักห้องนั้น
- 5.Room(ห้องพัก): เก็บข้อมูลของห้องพัก เช่น เลขห้อง และสถานะของห้องนั้นๆ
- 6.Payment (การชำระเงิน): ดำเนินการชำระเงินผ่านช่องทางต่างๆ

CLASS DIAGRAM



Flow process

ขั้นตอนที่ 1: การค้นหาและตรวจสอบ (Search & Check)

ลูกค้าเลือกประเภทห้องและวันที่ต้องการเข้าพัก

- 1.System รับข้อมูลจากหน้าเว็บ (ประเภทห้อง, วันที่)
- 2.เรียกใช้ RoomInventory.availableCount: ระบบจะไปดึงข้อมูลจาก Class RoomInventory ของวันที่ระบุมาดูว่าห้องประเภทนั้นเหลือเท่าไร
- 3.แสดงผล: ถ้า availableCount > 0 หน้าเว็บจะแสดงปุ่ม "จองห้องพัก" ให้ลูกค้ากด

ขั้นตอนที่ 2: การกักห้องชั่วคราว (The Locking Phase)

จังหวะที่ลูกค้ากดปุ่ม "จอง" คือการวัดดวงว่าใครเร็วกว่า

- 1.เรียกใช้ RoomInventory.tryAcquireLock():
 - ระบบจะเช็คใน Database ทันทีว่าห้องยังว่างไหม ถ้าว่างจะลด availableCount ลง 1 ทันที เพื่อไม่ให้คนอื่นมาจองซ้ำ
- 2.เรียกใช้ Constructor ของ Booking:
 - ระบบสร้าง Object Booking ขึ้นมาใหม่
 - กำหนด status = PENDING_LOCK
 - บันทึกเวลาที่กักใน lockedAt
 - คำนวณเวลาหมดอายุใน expiresAt (เช่น เวลาปัจจุบัน + 15 นาที)

ขั้นตอนที่ 3: การชำระเงิน (Payment Processing)

ขณะที่ลูกค้ากำลังกรอกข้อมูลบัตรเครดิต ห้องจะถูกกักไว้ให้ลูกค้ารายนี้คนเดียว

- 1.เรียกใช้ Payment.processPayment(): เมื่อลูกค้ากรอกข้อมูลเสร็จและกดจ่ายเงิน ระบบจะเรียกใช้คลาส Payment เพื่อส่งข้อมูลไปยังธนาคาร
- 2.ตรวจสอบผล:
 - ถ้าสำเร็จ (Success): ระบบจะเรียก Booking.confirm() เพื่อเปลี่ยนสถานะเป็น CONFIRMED เป็นอันเสร็จสิ้นการจอง
 - ถ้าล้มเหลว (Failed): ระบบจะแจ้งเตือนลูกค้าให้ลองใหม่อีกครั้ง (สถานะ Booking ยังคงเป็น PENDING_LOCK จนกว่าจะหมดเวลา)

ขั้นตอนที่ 4: การคืนห้องหรือยกเลิก (Timeout/Release)

กรณีที่ลูกค้ากักห้องไว้แล้วไม่จ่ายเงิน หรือปิดหน้าจอหนีไป

1.เรียกใช้ Booking.handleTimeout():

-ระบบหลังบ้านคอยเช็คว่ามี Booking ไหนที่สถานะเป็น PENDING_LOCK และเวลาปัจจุบันเลย expiresAt ไปแล้วหรือยัง

2.เรียกใช้ RoomInventory.releaseLock():

-หากหมดเวลา ระบบจะเปลี่ยนสถานะ Booking เป็น EXPIRED และสั่งให้ RoomInventory เพิ่มค่า availableCount กลับคืนมา 1 ห้อง เพื่อให้ลูกค้าคนอื่นสามารถจองได้ใหม่

4.System Architecture

4.1 ภาพรวมระบบ (System Overview)

ระบบ Hotel Booking System ออกแบบมาเพื่อรองรับการค้นหาห้องพัก จองห้อง และชำระเงินออนไลน์ โดยแก้ปัญหาหลักคือการจองห้องพักพร้อมกันจากหลาย channel โดยไม่ให้เกิดการจองซ้ำ (Double Booking)

Architecture แบ่งออกเป็น 6 Layer ที่มีหน้าที่แยกกันชัดเจน (Separation of Concerns) และสื่อสารกันผ่าน protocol มาตรฐาน

Layer	ชื่อ	หน้าที่หลัก
L1	Client & Traffic Control	รับ request จากผู้ใช้ กรอง traffic อันตราย กระจาย load
L2	Application Logic	Business logic หลักทั้งหมด แบ่งเป็น 7 Microservices
L3	Cache Layer	เก็บข้อมูลชั่วคราว ลด DB load และทำ Distributed Lock
L4	Database Layer	เก็บข้อมูลถาวร รองรับ ACID transactions
L5	External Services	บริการภายนอก เช่น Payment Gateway
L6	Observability	Monitor metrics, logs, alerts ของระบบ

4.2 รายละเอียดแต่ละ Layer

L1 — Client Layer & Traffic Control

Layer แรกรับ request จากผู้ใช้และกรองก่อนส่งต่อ Backend ทุก request ต้องผ่าน 3 ด้านตามลำดับ

Web App	React / Next.js — UI สำหรับผู้ใช้งาน Browser ส่ง request ผ่าน HTTPS เท่านั้น
Mobile App	iOS / Android — UI บนมือถือ ใช้ API endpoint เดียวกับ Web
WAF	กรอง SQL Injection, XSS, DDoS และบังคับ TLS 1.3 ก่อน traffic เข้าระบบ
Load Balancer	Nginx / AWS ALB — กระจาย traffic ด้วย round-robin พร้อม health check อัตโนมัติ
API Gateway	ตรวจสอบ JWT, ทำ Rate Limiting และ routing ไปยัง service ที่ถูกต้อง

L2 — Application Logic (Microservices)

Business logic ทั้งหมดอยู่ใน Layer นี้ แบ่งเป็น 7 Microservices โดยแต่ละตัวมีหน้าที่เดียว (Single Responsibility)

Auth Service	Register, Login, ออก JWT access/refresh token, JWT blacklist เมื่อ logout
Room Service	ค้นหาห้องว่าง, คำนวณราคา (getPricing), ดึงรูปภาพจาก S3
Booking Service	Core ของระบบ — จัดการ booking lifecycle: PENDING_LOCK → CONFIRMED / EXPIRED
Inventory Service	ควบคุม availableCount ด้วย atomic SQL เพื่อป้องกัน Double Booking
Payment Service	เชื่อมต่อ Omise/Stripe, รับ Webhook callback, รองรับ Refund
Notification Svc	ส่ง Email / SMS / Push Notification รับงานผ่าน Message Queue เท่านั้น
Message Queue	RabbitMQ / SQS — async event bus และรัน Timeout Worker ทุก 1 นาที

L3 — Cache Layer

Redis	ทำงาน 3 บทบาท: (1) Session + JWT blacklist (2) Cache availableCount พร้อม TTL (3) Distributed Lock สำหรับกัน race condition
-------	---

L4 — Database Layer

PostgreSQL Master	Primary data store รับ write ทั้งหมด เก็บ Guest, Booking, RoomInventory, Payment รองรับ ACID transactions
Read Replicas	สำเนาของ Master สำหรับ read-only queries เช่น ค้นหาห้องพัก ลด load บน Master
Object Store (S3)	เก็บรูปภาพห้องพักและเอกสาร แยกจาก DB เพื่อประสิทธิภาพและประหยัดค่าใช้จ่าย

L5 — External Services

Payment Gateway	Omise / Stripe — ประมวลผลการชำระเงินกับธนาคาร ส่ง Webhook result กลับมาเมื่อได้ผล
-----------------	---

L6 — Observability

Prometheus	Scrape metrics จากทุก service ทุก interval, trigger alert ผ่าน Alertmanager
Grafana	แสดง dashboard real-time, ติดตาม SLO/SLA และ alert routing
ELK Stack	Logstash รับ log stream, Elasticsearch index, Kibana ค้นหาและ visualize logs

4.3. การแก้ปัญหา Double Booking (Concurrency Control)

ปัญหาหลักของระบบคือ "ถ้ามีคนจองห้องเดิมพร้อมกันจากหลาย channel จะเกิดอะไรขึ้น?" ระบบแก้ด้วย 2 ชั้น:

ชั้นที่ 1 — Redis Distributed Lock

- เมื่อ Booking Service instance หลายตัวรับ request พร้อมกัน
- Redis SET NX (Set if Not eXists) ตัดสินว่าใครได้ lock ก่อน
- Lock มี TTL อัตโนมัติ ถ้า service crash — lock หายเองโดยไม่ต้อง cleanup

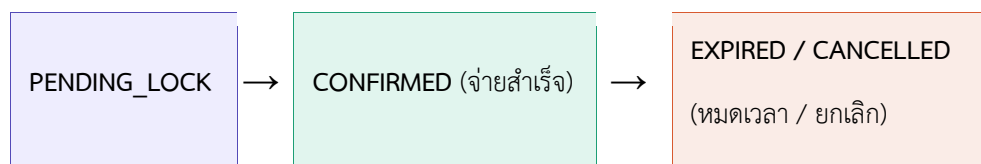
ชั้นที่ 2 — Atomic SQL ใน Inventory Service

Inventory Service รัน query แบบ atomic:

```
UPDATE room_inventory SET available_count = available_count - 1  
WHERE type_id = ? AND date = ? AND available_count > 0
```

- ถ้า affected rows = 1 → lock สำเร็จ สร้าง Booking (PENDING_LOCK)
- ถ้า affected rows = 0 → ห้องเต็ม ตีกลับ 409 Conflict

Booking Status Lifecycle



4.4 ความสัมพันธ์ระหว่าง Components

ตารางด้านล่างแสดงทุกเส้นเชื่อมใน Architecture Diagram พร้อมคำอธิบายว่าแต่ละเส้นทำอะไรและเพื่ออะไร

จาก	ไปยัง	สิ่งที่ส่ง / Label	เพื่ออะไร
Web / Mobile	WAF	HTTPS request	ส่ง request เข้าระบบผ่าน encrypted channel
WAF	Load Balancer	Clean traffic	กรอง request อันตรายออกก่อนเข้า backend
Load Balancer	API Gateway	API Request	กระจาย load ไปหลาย instance
API Gateway	Auth Service	Verify JWT	ตรวจสอบว่าผู้ใช้ login อยู่และมีสิทธิ์
Auth Service	API Gateway	valid / invalid	ตัดสินใจว่า route ต่อหรือตีกลับ 401
API Gateway	Room Service	Route GET /rooms	ส่งไปหา service ที่รับผิดชอบห้องพัก
API Gateway	Booking Service	Route POST /bookings	ส่งไปหา service ที่รับผิดชอบการจอง
API Gateway	Payment Service	Route POST /payments	ส่งไปหา service ที่รับผิดชอบการชำระเงิน
Auth Service	Redis	Store session / blacklist	เก็บ session และ token ที่ถูก logout ไว้ ตรวจสอบได้เร็ว
Auth Service	PostgreSQL	Read/Write Guest	บันทึกและดึงข้อมูลผู้ใช้ที่เป็น source of truth
Auth Service	Notification Svc	Welcome email event	แจ้ง Notification ส่ง email หลัง register แบบ async
Room Service	Redis	Cache check hit/miss	ลด DB round-trips ตอบได้เร็วยถ้ามี cache
Room Service	Read Replicas	Query rooms	ดึงข้อมูลห้องโดยไม่กระทบ Master
Room Service	S3	Fetch room images	ดึงรูปภาพที่เก็บแยกจาก DB
Room Service	Booking Service	availableCount	ให้ Booking รู้ว่าห้องว่างพอ lock ได้ไหม
Booking Service	Inventory Service	tryAcquireLock	ลด availableCount แบบ atomic กัน Double Booking

Room Service	Inventory Service	releaseLock	คืนห้องเมื่อ booking expire หรือถูกยกเลิก
Booking Service	Redis	Distributed lock	กัน race condition ระหว่าง Booking instances
Booking Service	Payment Service	processPayment	ส่งตัดเงินผ่าน Payment Gateway
Booking Service	Message Queue	Publish booking.created	ส่ง event async กัน block response ทำ user
Booking Service	PostgreSQL	Read/Write Booking	บันทึกทุก booking ให้ persistent และ query ได้
Payment Service	Message Queue	Publish payment.success	แจ้งผล async ให้ Booking update status
Payment Service	PostgreSQL	Read/Write Payment	บันทึกประวัติชำระเงินสำหรับ audit trail
Payment Service	Payment Gateway	Charge card	ประมวลผลกับธนาคารโดยไม่ต้องจัดการ PCI เอง
Payment Gateway	Payment Service	Webhook result	รับผลการชำระที่มาจากฝั่ง เช่น 3D Secure
Inventory Svc	PostgreSQL	Read/Write Inventory	บันทึก availableCount ให้ตรงกับความ เป็นจริง
Message Queue	Notification Svc	Consume event	รับงานส่ง email/SMS แบบ async ไม่ บล็อก booking
Message Queue	Booking Service	releaseLock (timeout)	expire booking ที่ไม่ชำระใน 15 นาที คืน ห้องอัตโนมัติ
PostgreSQL	Read Replicas	Replicate	ให้ Replica มีข้อมูล up-to-date รับ read query แทน Master
Booking Service	Prometheus	Metrics	ติดตาม error rate และ latency แบบ real-time
Auth / Pay Svc	ELK Stack	Log stream	เก็บ log สำหรับ debug และ audit trail
Prometheus	Grafana	Query metrics	ดึงข้อมูล metrics มาแสดงเป็น dashboard

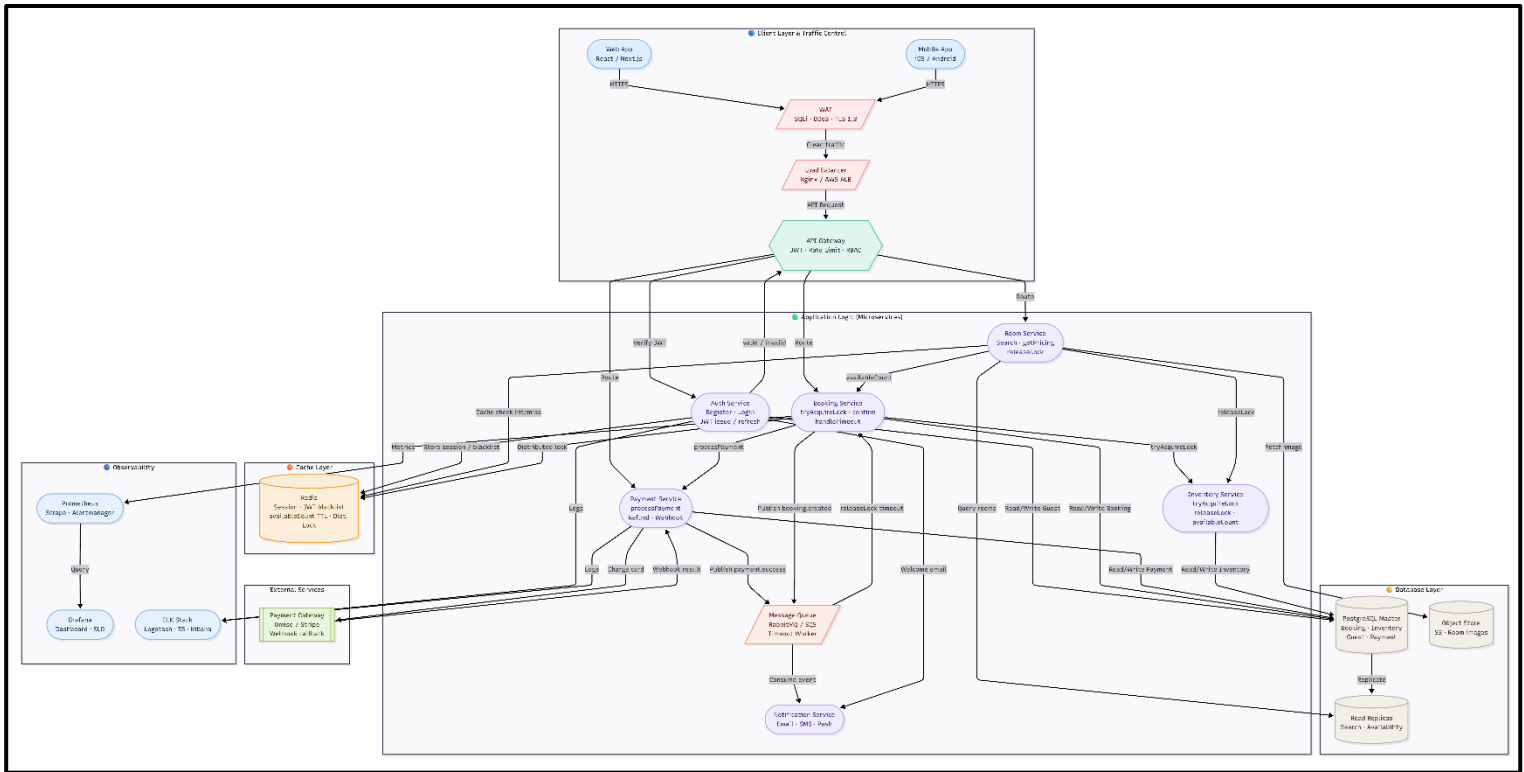
4.6 Happy Path — ขั้นตอนการจองแบบสมบูรณ์

Step	Component	การทำงาน
1	Client → WAF → LB → GW	Guest ค้นหาห้องผ่าน HTTPS → WAF กรอง → LB กระจาย → API Gateway ตรวจสอบ JWT
2	Room Service + Redis	ดึง availableCount จาก Redis Cache (hit) หรือ Read Replicas (miss) → ส่งรายการห้องกลับ
3	Booking Service + Redis	Guest กดจอง → Redis Distributed Lock → Inventory Service tryAcquireLock() (atomic SQL)
4	Booking Service + DB	Lock สำเร็จ → สร้าง Booking (PENDING_LOCK, expiresAt = +15 นาที) → บันทึก PostgreSQL
5	Payment Service + Gateway	Guest จ่ายเงิน → Payment Service → Omise/Stripe → ได้รับผลทันที หรือผ่าน Webhook
6	Booking Service + MQ	Payment สำเร็จ → confirm() → CONFIRMED → Publish booking.confirmed เข้า Message Queue
7	Notification Service	Consume event → ส่ง Email/SMS ยืนยันการจองให้ Guest

Timeout Path — เมื่อไม่ชำระภายใน 15 นาที

- Timeout Worker ใน Message Queue รันทุก 1 นาที
- ตรวจสอบ Booking ที่ status = PENDING_LOCK และ expiresAt < now()
- handleTimeout() → เปลี่ยน status เป็น EXPIRED → บันทึก PostgreSQL
- releaseLock(1) → Inventory Service เพิ่ม available_count กลับ → ห้องพร้อมจองใหม่
- Publish booking.expired → Notification Service ส่งแจ้ง Guest

SystemArchitecture DIAGRAM



5.Sequence Diagram

Business Logic ที่เลือกคือ **Atomic Inventory Locking**(การกักห้องพักแบบอะตอมมิก)

ทำไมถึงใช้ชื่อนี้: คำว่า "Atomic" หมายถึงการที่ระบบ "เชื่อว่ามีค่า" และ "หักจำนวนห้อง" ต้องเกิดขึ้นในขั้นตอนเดียวแบบแยกส่วนไม่ได้ เพื่อป้องกันไม่ให้ใครแทรกกลางได้

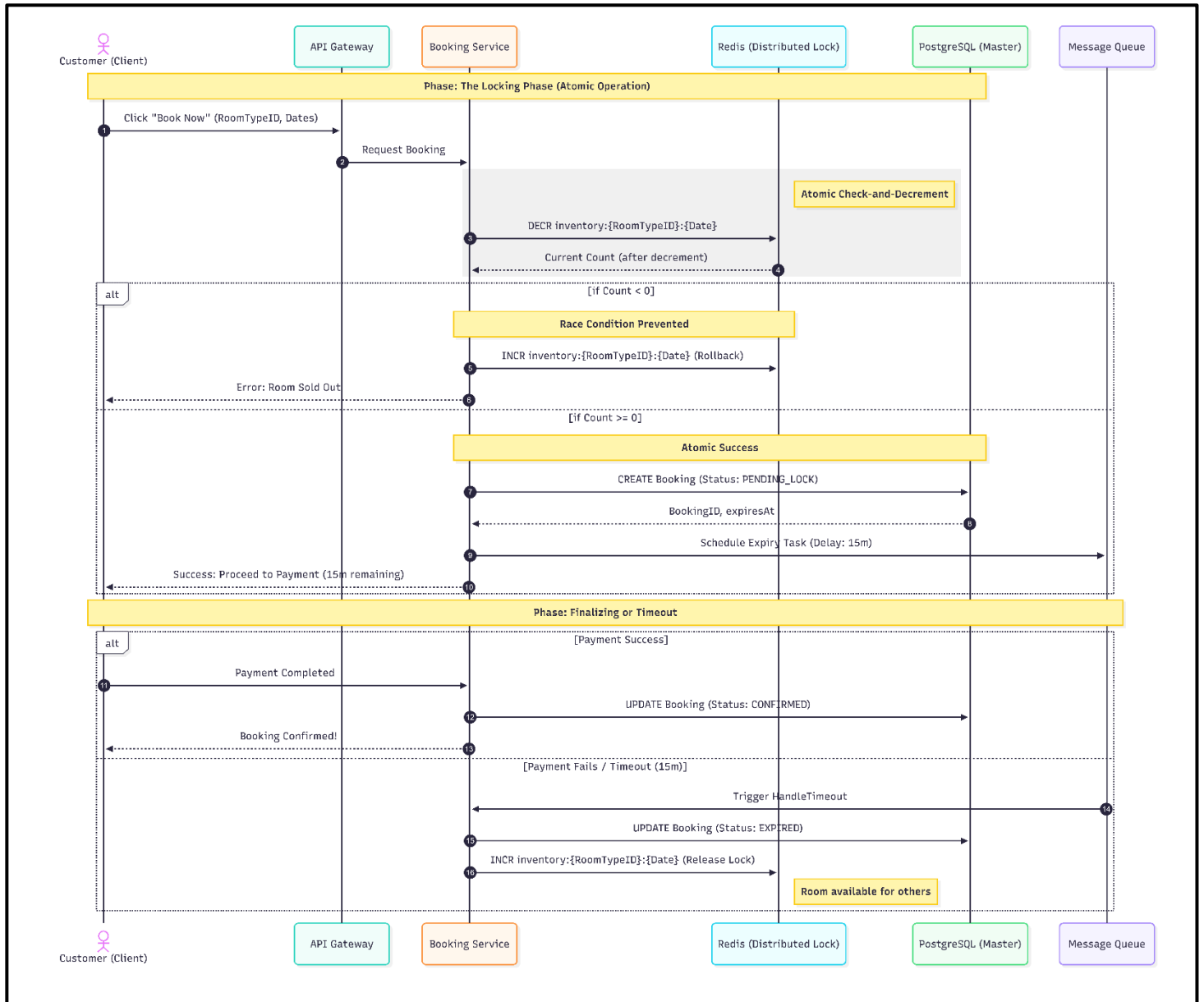
อธิบาย Flow ตามลำดับ

1. ผู้ใช้งาน (Customer) กด “Book Now” พร้อมระบุ RoomTypeID และวันที่ต้องการจอง
2. คำขอถูกส่งผ่าน API Gateway ไปยัง Booking Service
3. Booking Service ทำการเรียก Redis เพื่อหักจำนวนห้องด้วยคำสั่ง DECR inventory:{RoomTypeID}:{Date} ซึ่งเป็น atomic operation
4. Redis ส่งค่าจำนวนห้องหลังจากลดแล้วกลับมาให้ Booking Service
5. กรณีที่จำนวนห้องน้อยกว่า 0 (Count < 0)
 - o ระบบพิจารณาว่าห้องเต็มแล้ว
 - o Booking Service ทำการ rollback โดยใช้คำสั่ง INCR เพื่อคืนค่าจำนวนห้อง
 - o ระบบส่งข้อความแจ้งผู้ใช้งานว่าไม่สามารถจองได้ (Room Sold Out)
6. กรณีที่จำนวนห้องมากกว่าหรือเท่ากับ 0 (Count ≥ 0)

- ระบบสามารถดำเนินการจองต่อได้
 - Booking Service สร้างรายการจองในฐานข้อมูล PostgreSQL โดยกำหนดสถานะเป็น PENDING_LOCK
 - ระบบกำหนดเวลา expiration และส่งงานไปยัง Message Queue เพื่อจัดการ timeout (เช่น 15 นาที)
 - ระบบแจ้งผู้ใช้งานให้ดำเนินการชำระเงินภายในเวลาที่กำหนด
7. เมื่อผู้ใช้งานชำระเงินสำเร็จ
- Booking Service อัปเดตสถานะการจองเป็น CONFIRMED
 - การจองเสร็จสมบูรณ์
8. กรณีผู้ใช้งานไม่ชำระเงินภายในเวลาที่กำหนด (timeout)
- Message Queue เรียกกระบวนการ HandleTimeout
 - Booking Service อัปเดตสถานะเป็น EXPIRED
 - ระบบคืนจำนวนห้องกลับไปยัง Redis ด้วยคำสั่ง INCR (release lock)
 - ห้องจะกลับมาอยู่ในสถานะที่สามารถจองได้อีกครั้ง

Sequence Diagram นี้แสดงกระบวนการจองห้องพักโดยใช้แนวคิด Atomic Inventory Locking เพื่อป้องกันปัญหา overbooking ในกรณีที่ผู้ใช้หลายคนทำรายการพร้อมกัน โดยระบบใช้ Redis ในการทำ atomic operation สำหรับการลดจำนวนห้อง (DECR) ซึ่งทำให้การตรวจสอบจำนวนห้องว่างและการหักจำนวนเกิดขึ้นในขั้นตอนเดียว ไม่สามารถมีคำสั่งอื่นแทรกกลางได้ นอกจากนี้ยังมีการใช้ฐานข้อมูลสำหรับบันทึกสถานะการจอง และ Message Queue สำหรับจัดการกรณีหมดเวลาชำระเงินและคืนห้องกลับเข้าสู่ระบบ

Sequence Diagram



6. ประเด็นที่ระบบครอบคลุม

6.1 Scalability:

Traffic Control: การมี Load Balancer และ API Gateway ช่วยให้ทำ Horizontal Scaling ในส่วนของ Microservices ได้อย่างอิสระ เมื่อมีคนจองเยอะก็สามารถสั่งเพิ่มแค่ Booking Service หรือ Inventory Service ได้โดยไม่กระทบส่วนอื่น

Database Scaling: มี Read Replicas เพราะในระบบจองโรงแรม จำนวนคน "ค้นหา" จะมากกว่าคน "จอง" เสมอ การแยก Traffic ค้นหาไปที่ Replica ช่วยลดภาระได้มหาศาล

6.2 Data Strategy:

PostgreSQL: เหมาะสมที่สุดสำหรับข้อมูลธุรกรรม (Booking, Payment) เพราะต้องการความแม่นยำ (ACID)

Redis & Concurrency: วาง Redis ไว้ก่อนถึง Inventory Service เพื่อทำ Distributed Lock เป็นการแก้ปัญหา "จองซ้ำ" (Double Booking) และความเร็วของ Redis ช่วยให้ tryAcquireLock ทำงานได้อย่างรวดเร็ว

S3: การแยกรูปภาพห้องพักออกไปที่ Object Storage ช่วยให้ Database ไม่บวมและโหลดข้อมูลได้เร็วขึ้น

6.3 Communication:

Message Queue: ใช้ Message Queue เชื่อมโยงไปยัง Notification Service และการจัดการ releaseLock ช่วยให้ระบบทำงานแบบ Asynchronous ลูกค้าไม่ต้องรอให้ Email ส่งเสร็จถึงจะเห็นหน้าจอยืนยันการจอง

API Gateway: การ Verify JWT ที่ Gateway จะช่วยลดภาระของ Microservices ด้านใน ไม่ต้องตรวจสอบซ้ำซ้อนในทุก Request

6.4 Resiliency:

WAF: ป้องกันการโจมตีระดับ Application (SQLi, DDoS) ได้ดี

Observability: มี Prometheus, Grafana และ ELK Stack ช่วยให้ถ้าหาก Service ใดพัง (เช่น จองสำเร็จแต่ตัดเงินไม่ผ่าน) จะสามารถใช้ Distributed Tracing ไล่ดู Log ได้ทันทีว่าพังที่จุดไหน

Fault Tolerance: ระบบ Message Queue ช่วยให้มั่นใจว่าถ้า Notification Service ล่มชั่วคราว ข้อความจะไม่หาย เมื่อ Service กลับมาออนไลน์ก็จะดึงไปทำงานต่อได้เอง

7.C4 Model: Hotel Booking System

แนวคิดของ C4 Model

C4 Model เป็นรูปแบบการอธิบายสถาปัตยกรรมระบบซอฟต์แวร์ โดยแบ่งมุมมองของระบบออกเป็นหลายระดับ เพื่อให้เข้าใจง่าย ตั้งแต่ภาพรวมระดับผู้ใช้งานไปจนถึงส่วนประกอบภายในของระบบ โดยในระบบจองห้องพักโรงแรมนี้จะใช้ C4 Model ทั้งหมด 4 ระดับ ได้แก่

Level 1: System Context Diagram

แสดงภาพรวมของระบบ ผู้ใช้งาน และระบบภายนอกที่เกี่ยวข้อง

Level 2: Container Diagram

แสดงองค์ประกอบหลักของระบบ เช่น Web App, API Gateway, Microservices, Database, Redis และ Message Queue

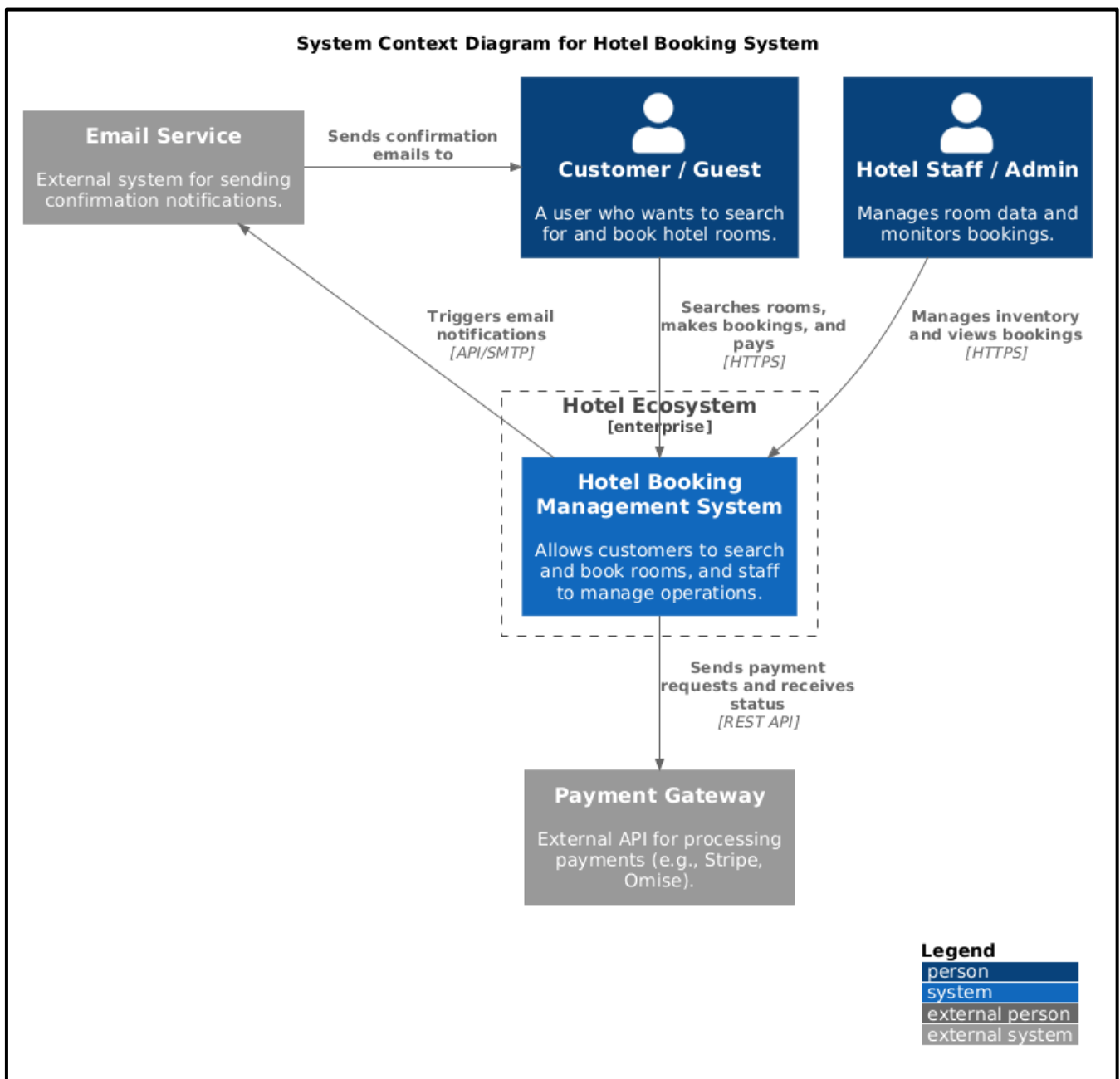
Level 3: Component Diagram

แสดงรายละเอียดภายใน Service สำคัญ โดยเฉพาะ Booking Service และ Inventory Service ซึ่งเป็นหัวใจของการป้องกันปัญหาการจองซ้ำ

Level 4: Code Diagram

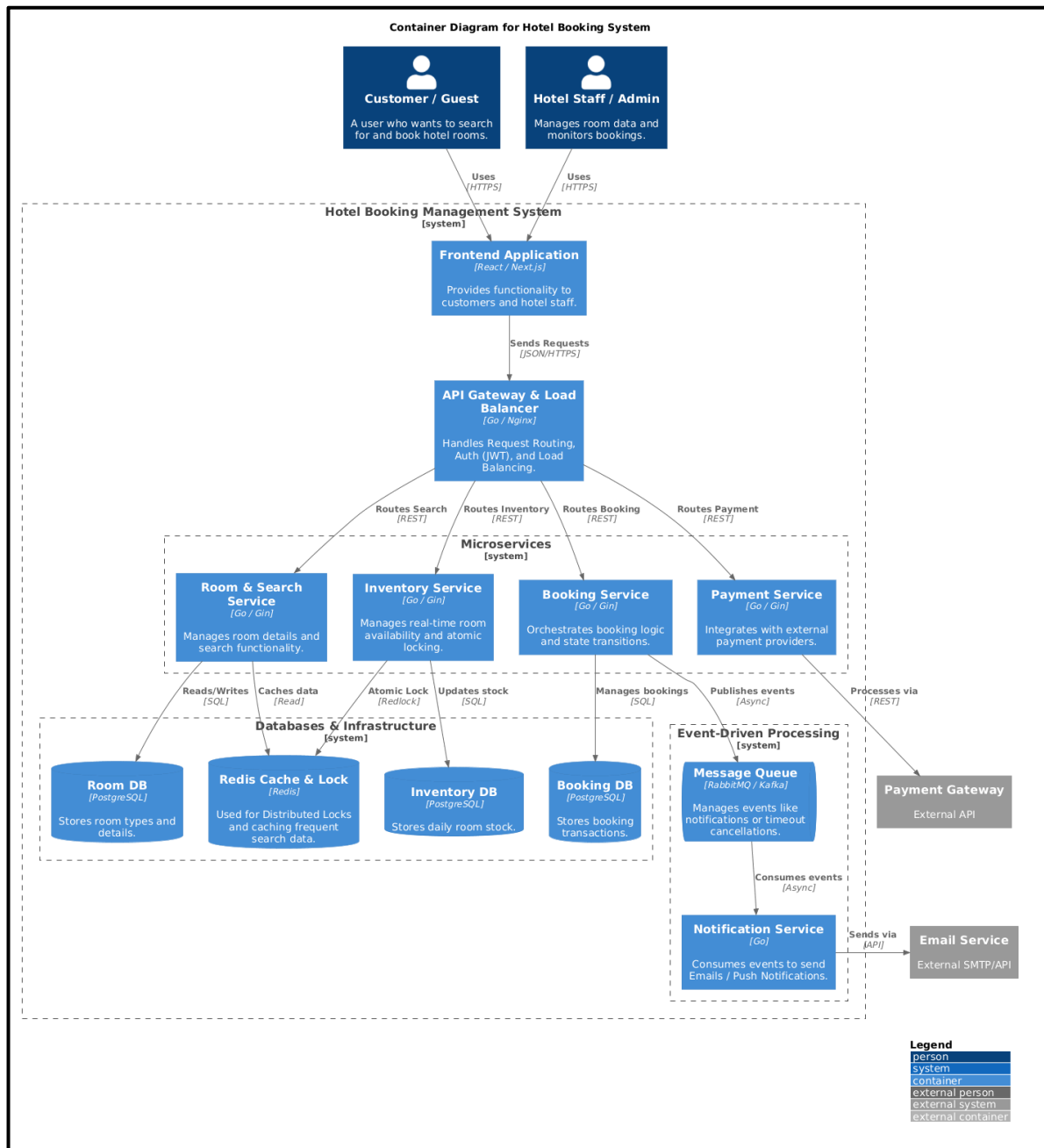
แสดงรายละเอียดในระดับโค้ดหรือคลาสภายใน Component สำคัญของระบบ โดยจะอธิบายว่าแต่ละคลาส เมธอด หรือ ฟังก์ชันทำงานร่วมกันอย่างไร

C4 Level 1: System Context Diagram



System Context Diagram แสดงภาพรวมระดับสูงสุดของระบบจัดการการจองห้องพัก (Hotel Booking Management System) แผนภาพนี้มุ่งเน้นแสดงปฏิสัมพันธ์ระหว่างผู้ใช้งานระบบและระบบภายนอก โดยมีผู้ใช้งานหลัก 2 กลุ่ม ได้แก่ ลูกค้า (Customer) ที่เข้ามาค้นหาและจองห้องพัก และพนักงาน (Admin) ที่เข้ามาจัดการข้อมูล นอกจากนี้ยังแสดงขอบเขตการเชื่อมต่อกับระบบภายนอก (External Dependencies) ที่อยู่นอกเหนือการควบคุมของระบบเรา ได้แก่ Payment Gateway สำหรับประมวลผลการชำระเงิน และ Email Service สำหรับจัดส่งการแจ้งเตือน ซึ่งช่วยให้ผู้ที่มีส่วนเกี่ยวข้อง (Stakeholders) เห็นภาพรวมของ Ecosystem ทั้งหมดได้อย่างรวดเร็ว

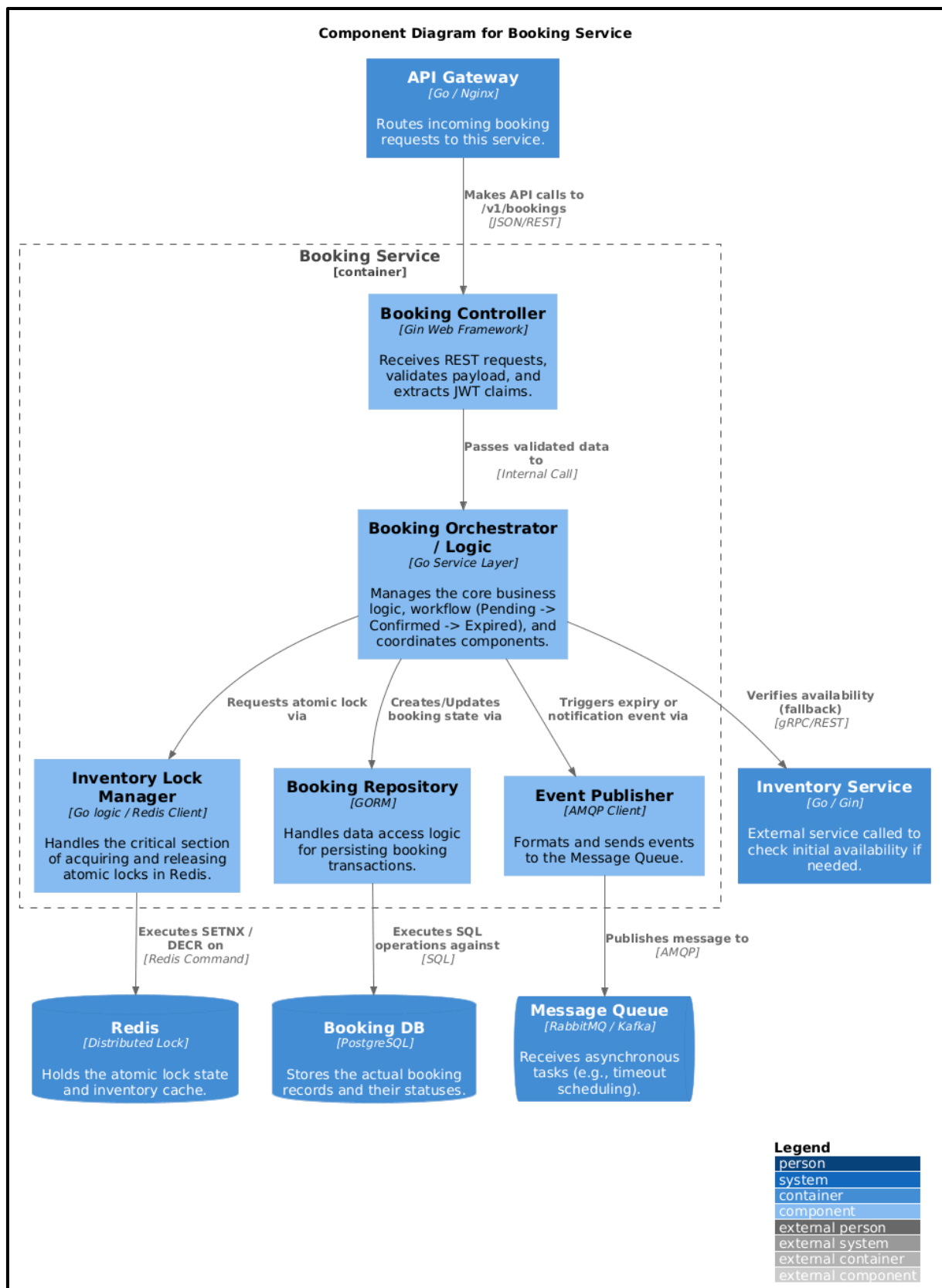
C4 Level 2: Container Diagram



Container Diagram แสดงโครงสร้างทางเทคนิคและสถาปัตยกรรมภายในของระบบ โดยออกแบบบนพื้นฐานของ Microservices Architecture ระบบถูกแบ่งออกเป็น Service ย่อยๆ ที่ทำงานอิสระต่อกัน (เช่น Booking Service, Inventory Service) ซึ่งพัฒนาด้วยภาษา Go และใช้ Gin Framework ด้านหน้าของระบบมี API Gateway ทำหน้าที่ตรวจสอบสิทธิ์ (JWT Auth) และกระจายโหลด (Load Balancing)

ในด้านการจัดการข้อมูล ได้ประยุกต์ใช้แนวคิด Database per Service โดยใช้ PostgreSQL แยกอิสระสำหรับแต่ละบริการ เพื่อลดการพึ่งพากันระหว่างฐานข้อมูล นอกจากนี้ยังมีการบูรณาการ Redis เพื่อทำ Distributed Lock สำหรับแก้ปัญหา Concurrency ในการจองห้องพัก และใช้ Message Queue สร้างสถาปัตยกรรมแบบ Event-Driven เพื่อรองรับการทำงานแบบ Asynchronous ช่วยเพิ่มประสิทธิภาพการขยายตัว (Scalability) ของระบบ

C4 Level 3: Component Diagram — Booking Service

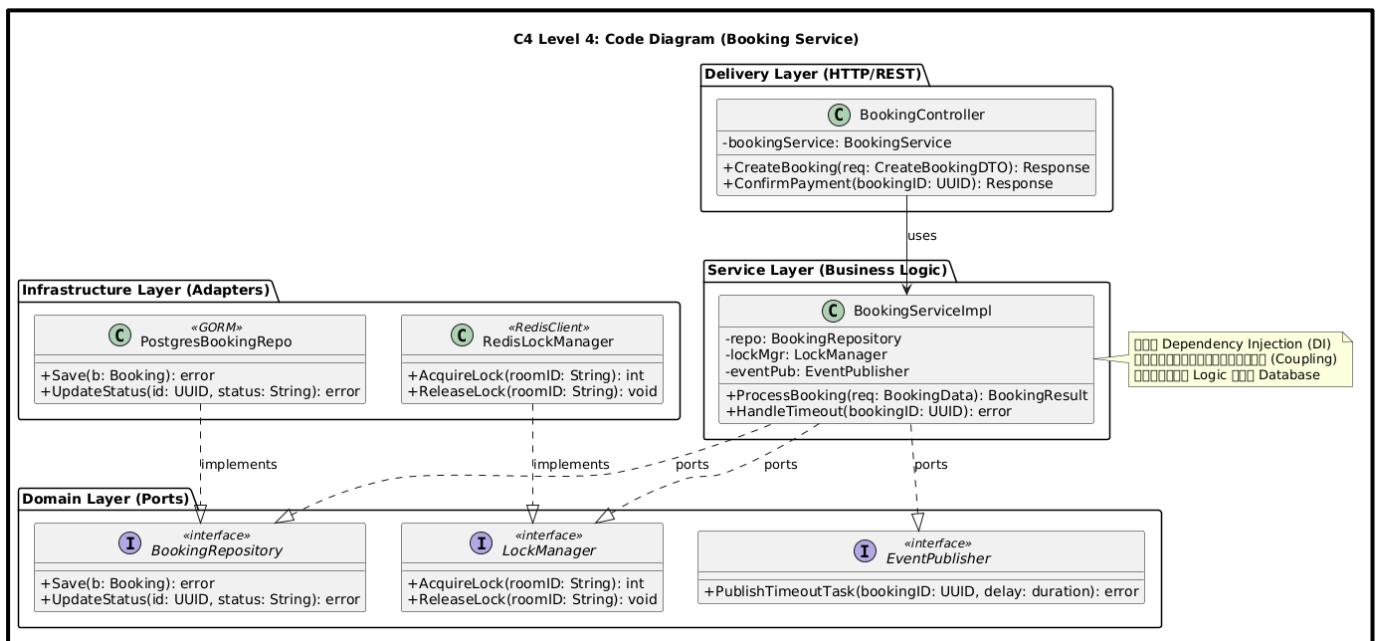


แผนภาพ Component Diagram นี้จะลึกเข้าไปใน Booking Service ซึ่งเป็นบริการหลักของระบบ ภายใน Service มีการออกแบบโครงสร้างโค้ดตามหลักการ Separation of Concerns (SoC) โดยแบ่งออกเป็นชั้นการทำงาน (Layers) อย่างชัดเจน

เมื่อ API Gateway ส่งคำขอเข้ามา Booking Controller จะทำการตรวจสอบข้อมูล (Validation) และส่งต่อไปยัง Booking Orchestrator ซึ่งเป็นศูนย์กลางควบคุมกระบวนการทางธุรกิจทั้งหมด จุดเด่นของสถาปัตยกรรมนี้คือการเรียกใช้ Inventory Lock Manager เพื่อทำ Atomic Distributed Lock ผ่าน Redis ก่อนทำการบันทึกข้อมูลลงฐานข้อมูลผ่าน Booking Repository (ใช้ GORM) วิธีนี้ช่วยแก้ปัญหา Race Condition และป้องกันการเกิด Overbooking ได้อย่างมีประสิทธิภาพ

ในขั้นตอนสุดท้าย Event Publisher จะส่งเหตุการณ์ที่เกิดขึ้น (เช่น เริ่มนับเวลาถอยหลัง 15 นาที หรือส่งอีเมลยืนยัน) ไปยัง Message Queue เพื่อให้ระบบสามารถประมวลผลต่อในรูปแบบ Asynchronous ได้ โดยไม่บล็อกการทำงานหลัก (Non-blocking) ตอบโจทย์ทั้งด้าน High Concurrency และ High Availability อย่างครบถ้วน

C4 Model Level 4: Code Diagram



Code Diagram (Class Diagram) ของ Booking Service ถูกออกแบบด้วยสถาปัตยกรรมแบบ Hexagonal Architecture (Ports and Adapters) หรือ Clean Architecture โดยมีการแยกส่วนชั้น Presentation (Controller), Business Logic (Service) และ Infrastructure (Database/Redis) ออกจากกันอย่างชัดเจน การใช้ Interface เป็นตัวกลางประสานงาน (Dependency Inversion Principle) ช่วยให้โมดูลต่างๆ ลดความผูกพัน (Loose Coupling) ส่งผลให้สามารถเขียน Unit Test สำหรับ Business Logic ได้ง่ายโดยไม่ต้องเชื่อมต่อกับฐานข้อมูลหรือ Redis จริง